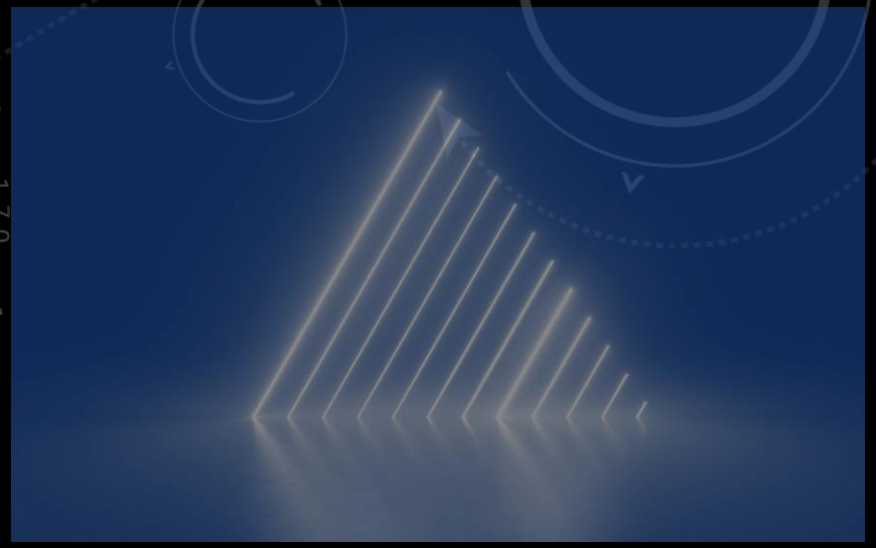


VARIABLE ARGUMENT LIST FUNCTIONS

WHAT IS IT ?

- A programmer may come across a situation, when a function, which can take variable number of arguments(parameters), instead of fixed number of parameters.

- In programming language C++ and Java this situation is handled with function overloading concept.

- The C programming language provides a solution for this situation by defining a function which can accept variable number of parameters based on your requirement.

- Indeed, some library functions can accept a variable list of arguments such as, printf(), scanf(), fprintf(),fscanf(), fcntl(),ioctl(),execlp(),semctl() etc.

- Whenever a function is declared to have an indeterminate number of arguments, in place of the last argument you should place an ellipsis (', ...').

- *int fcntl(int ,int, ...);*

HOW TRANSLATOR(COMPILER) TREATS IT?

- Partial parameter list can be terminated by the ellipsis notation, a comma followed by three periods (, ...), to indicate that there may be more arguments passed to the function, but no more information is given about them.

- Type checking is not performed on such arguments.

- At least one parameter must precede the ellipsis notation and the ellipsis notation must be the last token in the parameter list.

HOW TRANSLATOR(COMPILER) TREATS IT?

int fcntl(int,int,…);

In the above example

1. fcntl(0,1);//valid
2. fcntl(0,1,2,3);//valid
3. fcntl(0);//invalid
4. fcntl();//invalid

- int printf(const char*,…);
- In the above example:
  1. printf(“hello”);//valid
  2. printf(“%s%s%s%s”,”hello”,”hi”,”how”,”are you”);//valid
  3. printf();//invalid

WHAT WE NEED TO DO?

- To use this functionality, we need to include <stdarg.h>.
- The stdarg.h header defines several macros used to get the arguments in a function when the number of arguments is not known.
- Macros: va_start(), va_arg(), va_end(),
- Variables: typedef va_list
- The va_list type is a type suitable for use in accessing the arguments of a function with the stdarg macros. *void va_start(va_list v, last_arg);*
- It initializes v for use with the va_arg and va_end macros. last_arg is the last known fixed argument being passed to the function (the argument before the ellipsis). Note that va_start must be called before using va_arg and va_end.

STEPS TO CREATE YOUR VARIABLE ARGUMENT LIST FUNCTION

- Define a function with its last parameter as ellipses, and before the ellipses there should exist at least one fixed argument.
- Create a va_list type variable in the function definition. This type is defined in stdarg.h header file.
- Use last fixed parameter and va_start macro to initialize the va_list variable to an argument list. The macro va_start is defined in stdarg.h header file.
- Use va_arg macro and va_list variable to access each item in argument list.
- Use a macro va_end to clean up the memory assigned to va_list variable.

TWO WAYS OF IMPLEMENTATION.

1. passing number of variable arguments as fixed argument. E.g
   - *printStrings(2,"Vector", "India");*
   - *printStrings(5, "Vector", "india","pvt", "ltd","Hyderabad");*

2. passing null(special end data) at the end of arguments. E.g
   - printStrings("Vector","India",NULL);
   - printStrings("Vector","India","pvt","ltd","Hyderabad",NULL);

METHOD 1

```
#include<stdio.h>
#include<stdarg.h>
void printStrings(int,...);
int main()
{
        printStrings(2,"Vector","India");
        printStrings(5,"Vector","India","Private","Limited","Hyderabad");
return 0;
}
void printStrings(int cnt,...)
{
        char *temp
        va_list handle;
va_start(handle,cnt);
        while(cnt--)
        {
                temp=va_arg(handle,char*);
                puts(temp);
        }
va_end(handle);
return;
}
```

METHOD 2

```
#include<stdio.h>
#include<stdarg.h>
void printStrings(const char*,...);
int main()
{
        printStrings(”Vector”,”India”,NULL);
        printStrings(”Vector”,”India”,”Private”,”Limited”,”Hyderabad”,NULL);
return 0;
}
void printStrings(const char *ptr,...)
{
        va_list handle;
va_start(handle, ptr);
puts(ptr);
        while(ptr=va_arg(handle,char*))
        {
                puts(temp);
        }
va_end(handle);
return;
}
```

- Refer to the manual page of va_start, va_arg, va_end ..
- if you don’t have linux, still you can find the manual in Google, by typing “man va_start”, etc.
- More examples will be explained , in class, in detail.